

ELO329 - Diseño y Programación Orientados a Objetos

Herencia en C++

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

La herencia permite construir una clase nueva a partir de una clase existente.

La clase derivada reutiliza estado y comportamiento de la clase base, y agrega lo que la hace más específica.

En esta clase veremos la mecánica en C++ usando un ejemplo simple: `CMotor` y `CElectricMotor`.

La herencia modela una relación "es un tipo de", no solo una forma de ahorrar código.

Contenidos

1. Clase base y clase derivada
2. Sintaxis de herencia en C++
3. Herencia pública
4. Inicialización de la clase base
5. Llamadas a métodos de la clase base
6. Redefinición de métodos
7. Asignación y slicing
8. Miembros `protected`
9. Herencia pública, protegida y privada
10. Miembros que no se heredan

Punto de partida

Consideremos una clase que representa un motor.

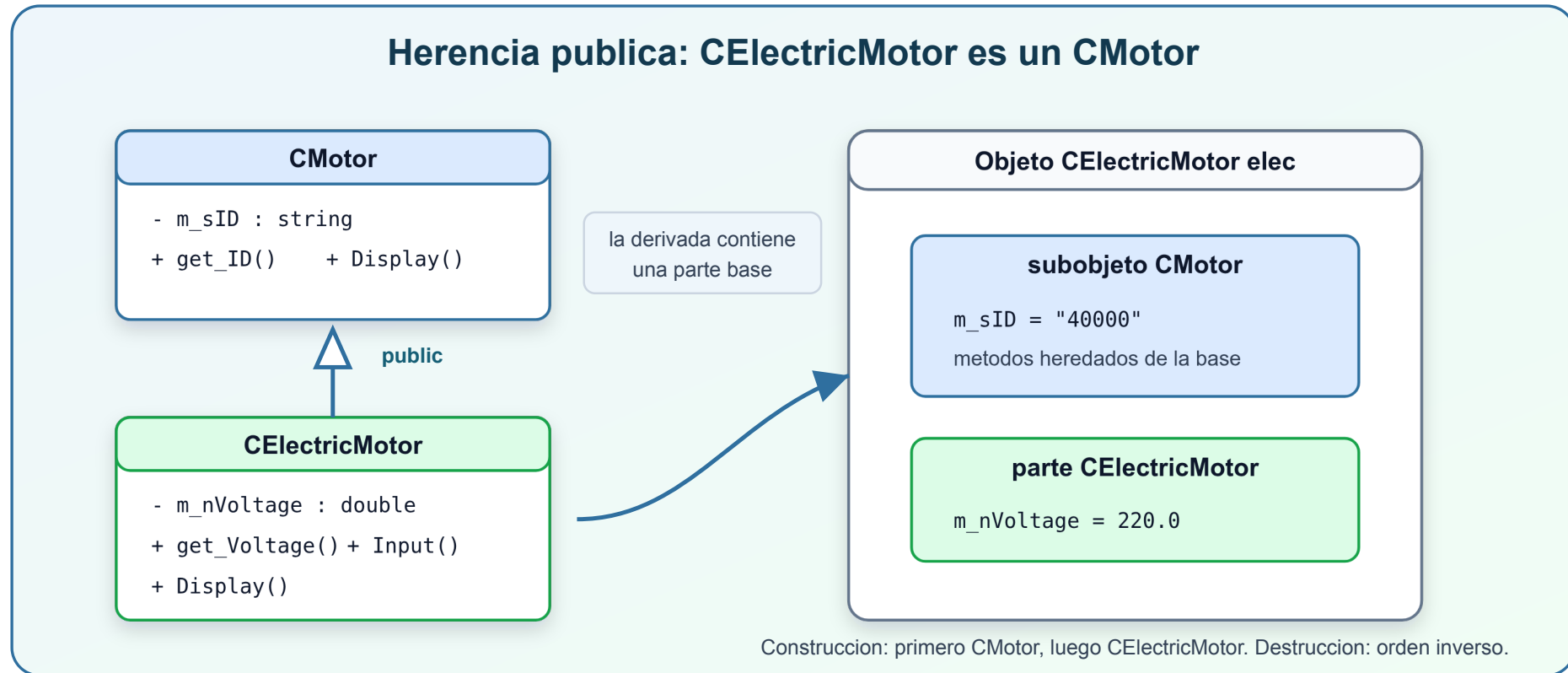
```
CMotor  
  ID
```

Un motor eléctrico también es un motor, pero agrega voltaje.

```
CElectricMotor  
  ID  
  voltage
```

La pregunta es cómo expresar esa relación sin duplicar el manejo del ID.

Relación conceptual



CElectricMotor contiene una parte CMotor y agrega su propio estado.

Por eso puede reutilizar operaciones comunes como `get_ID` , `set_ID` , `Input` y `Display` .

Clase base: CMotor

```
class CMotor {  
public:  
    CMotor() {}  
    CMotor(const std::string &id);  
  
    std::string get_ID() const;  
    void set_ID(const std::string &s);  
  
    void Display() const;  
    void Input();  
  
private:  
    std::string m_sID;  
};
```

Esta clase conoce todo lo necesario para administrar el ID de un motor.

Implementación de CMotor

```
CMotor::CMotor(const std::string &id) {  
    set_ID(id);  
}  
  
std::string CMotor::get_ID() const {  
    return m_sID;  
}  
  
void CMotor::set_ID(const std::string &s) {  
    m_sID = s;  
}
```

Los métodos públicos son la interfaz para leer y modificar el ID.

Display e Input

```
void CMotor::Display() const {  
    std::cout << "[CMotor] ID=" << m_sID;  
}  
  
void CMotor::Input() {  
    std::string temp;  
    std::cout << "Enter Motor ID: ";  
    std::cin >> temp;  
    set_ID(temp);  
}
```

Display observa el objeto, por eso es `const` .

Input modifica el ID, por eso no es `const` .

Crear una clase derivada

La sintaxis general es:

```
class Derivada : public Base {  
    // ...  
};
```

Ejemplo:

```
class CElectricMotor : public CMotor {  
    // ...  
};
```

La clase base aparece después de `:` junto al tipo de herencia.

Herencia pública

La herencia pública es la forma que representa mejor una relación "es un".

```
class CElectricMotor : public CMotor {  
    // ...  
};
```

Con esta relación:

- Un `CElectricMotor` es un `CMotor`.
- La interfaz pública de `CMotor` sigue siendo pública en `CElectricMotor`.
- Código que espera un `CMotor` puede trabajar con la parte base de un `CElectricMotor`.

Clase derivada: CElectricMotor

```
class CElectricMotor : public CMotor {  
public:  
    CElectricMotor();  
    CElectricMotor(const std::string &id, double volts);  
  
    void Display() const;  
    void Input();  
    void set_Voltage(double volts);  
    double get_Voltage() const;  
  
private:  
    double m_nVoltage;  
};
```

La clase derivada agrega voltaje y redefine operaciones de entrada y salida.

Constructor por defecto

```
CElectricMotor::CElectricMotor() {  
    m_nVoltage = 0;  
}
```

Antes de entrar al cuerpo del constructor derivado, C++ construye primero la parte base.

Aquí se invoca el constructor por defecto de `CMotor`.

Luego se ejecuta el cuerpo del constructor de `CElectricMotor`.

Inicializar la clase base

Para pasar argumentos al constructor de la clase base, usamos lista de inicialización.

```
CElectricMotor::CElectricMotor(const std::string &id,  
                                double volts)  
    : CMotor(id) {  
    m_nVoltage = volts;  
}
```

`CMotor(id)` construye la parte base del objeto.

Luego el cuerpo inicializa el estado propio de `CElectricMotor`.

Estructura general

```
Derived::Derived(args)
    : Base(args_base),
      member(args_member) {
    // cuerpo del constructor
}
```

Orden conceptual:

1. Se construye la clase base.
2. Se construyen los atributos de la derivada.
3. Se ejecuta el cuerpo del constructor derivado.

El orden de construcción importa cuando hay recursos o invariantes.

Llamar métodos de la base

La clase derivada puede reutilizar código de la base.

```
void CElectricMotor::Input() {  
    CMotor::Input();  
  
    double volts;  
    std::cout << "Voltage: ";  
    std::cin >> volts;  
    set_Voltage(volts);  
}
```

`CMotor::Input()` invoca explícitamente el método de la clase base.

Equivalente a super

En Java se usaría una idea similar con `super`.

```
super.input();
```

En C++ usamos el operador de resolución de alcance.

```
CMotor::Input();
```

Esto evita duplicar el código que ya sabe leer el ID del motor.

Display en la derivada

```
void CElectricMotor::Display() const {  
    CMotor::Display();  
  
    std::cout << " [CElectricMotor]"  
                << " Voltage=" << m_nVoltage  
                << std::endl;  
}
```

Primero se muestra la parte común del motor.

Luego se agrega la información específica del motor eléctrico.

Redefinición de métodos

CMotor y CElectricMotor pueden tener métodos con el mismo nombre.

```
CMotor mot("33333");  
CElectricMotor elec("40000", 220.0);  
  
mot.Display(); // CMotor::Display  
elec.Display(); // CElectricMotor::Display
```

Cuando usamos el objeto directamente, el tipo declarado del objeto determina qué método se llama.

Punteros a base

La herencia pública permite que un puntero a base apunte a un objeto derivado.

```
CMotor *p = new CElectricMotor("400000", 220.0);  
p->Display();
```

En este punto hay una pregunta importante:

¿Se llama Display de CMotor o de CElectricMotor?

Sin métodos `virtual`, C++ usa el tipo del puntero: `CMotor`.

Puente a ligado dinámico

El ejemplo anterior no está completo desde el punto de vista polimórfico.

```
CMotor *p = new CElectricMotor;  
p->Display(); // sin virtual: CMotor::Display
```

Para que la llamada use la versión de `CElectricMotor`, el método debe ser `virtual`.

Ese será el tema de la siguiente clase.

Hoy nos concentramos en herencia estructural. Luego veremos despacho dinámico.

Asignación derivada a base

C++ permite asignar un objeto derivado a un objeto base.

```
CMotor mot;  
CElectricMotor elec("40000", 220.0);  
  
mot = elec;
```

Solo se copia la parte `CMotor` de `elec` .

El voltaje no tiene dónde quedar en `mot` .

Object slicing

La pérdida de la parte derivada se conoce como slicing.

```
mot = elec;
```

Después de la asignación:

- `mot` conserva el ID copiado desde la parte base.
- `mot` no tiene `m_nVoltage`.
- `mot.get_Voltage()` no existe.

El objeto base no se transforma en eléctrico.

Asignación base a derivada

La dirección contraria no está permitida por defecto.

```
CMotor mot("33333");  
CElectricMotor elec;  
  
elec = mot; // error
```

Falta información para construir la parte específica de `CElectricMotor`.

El objeto base no tiene voltaje.

Miembros protected

Un miembro `protected` puede ser usado por la clase y por sus derivadas.

```
class CMotor {  
protected:  
    std::string get_ID() const;  
    void set_ID(const std::string &s);  
};
```

Desde fuera de la jerarquía, se comporta como no público.

Desde una clase derivada, se puede acceder.

private versus protected

Acceso	Misma clase	Clase derivada	Código externo
private	Sí	No	No
protected	Sí	Sí	No
public	Sí	Sí	Sí

`protected` abre una puerta para las derivadas sin convertir el miembro en parte de la interfaz pública.

Debe usarse con cuidado: aumenta el acoplamiento dentro de la jerarquía.

Tipos de herencia

C++ permite tres tipos de herencia.

```
class D1 : public B {};  
class D2 : protected B {};  
class D3 : private B {};
```

El tipo de herencia controla cómo se ven los miembros heredados desde fuera de la clase derivada.

Si se omite en una `class`, el valor por defecto es `private`.

Efecto del tipo de herencia

En la base	<code>public</code> inheritance	<code>protected</code> inheritance	<code>private</code> inheritance
<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>
<code>protected</code>	<code>protected</code>	<code>protected</code>	<code>private</code>
<code>private</code>	No accesible directo	No accesible directo	No accesible directo

La herencia pública conserva la interfaz.

La protegida y la privada esconden la relación hacia usuarios externos.

Herencia protegida

```
class CElectricMotor : protected CMotor {  
    // ...  
};
```

Los métodos públicos de `CMotor` pasan a ser protegidos en `CElectricMotor`.

```
CElectricMotor em;  
em.set_ID("12345"); // error
```

La clase derivada puede usar `set_ID`, pero el usuario externo no.

Herencia privada

```
class CElectricMotor : private CMotor {  
    // ...  
};
```

La clase derivada puede usar la base internamente, pero no expone la relación como interfaz pública.

Además, si otra clase deriva desde `CElectricMotor`, ya no recibe acceso a los miembros de `CMotor`.

Esto modela una relación de implementación, no una relación "es un".

Miembros que no se heredan

Algunos elementos no se heredan como métodos normales:

- Constructores.
- Destructor.
- Operador de asignación.
- Relaciones `friend`.

La clase derivada debe definir cómo se construye, destruye y asigna su propia parte.

La parte base igual se construye y destruye automáticamente como subobjeto.

Orden de destrucción

La destrucción ocurre en el orden inverso a la construcción.

Construcción:

CMotor

CElectricMotor

Destrucción:

CElectricMotor

CMotor

Esto asegura que la parte derivada se limpie antes de destruir la parte base que podría estar usando.

struct y class

En C++, `struct` también puede tener métodos, constructores y herencia.

La diferencia principal por omisión es el acceso:

Palabra	Acceso por omisión
<code>class</code>	<code>private</code>
<code>struct</code>	<code>public</code>

En diseño orientado a objetos normalmente se usa `class` para tipos con invariantes y encapsulamiento.

Herencia múltiple

C++ permite derivar desde más de una clase base.

```
class D : public B1, public B2 {  
    // ...  
};
```

Esto es una diferencia con Java.

También introduce ambigüedades y problemas de diseño que requieren cuidado.

En esta clase trabajaremos con herencia simple.

Conexión con diseño

Usar herencia pública cuando la relación sea de sustitución.

```
Un CElectricMotor es un CMotor.
```

Evitar herencia pública si solo queremos reutilizar implementación.

En ese caso, composición suele ser una alternativa más clara.

```
class Controller {  
private:  
    CMotor motor;  
};
```

Ideas clave

- Una clase derivada reutiliza y extiende una clase base.
- La herencia pública modela una relación "es un".
- La clase base se construye antes que la derivada.
- La lista de inicialización permite invocar constructores de la base.
- `Base::metodo( )` llama explícitamente una función de la clase base.
- Asignar derivada a base copia solo la parte base: slicing.
- `protected` permite acceso desde derivadas sin exponer al exterior.
- El tipo de herencia cambia la visibilidad heredada.
- Constructores, destructores, asignación y `friend` no se heredan como métodos normales.
- Para llamadas polimórficas mediante punteros a base se necesitan métodos `virtual`.